25-2 AI NLP

LLM-Based Category Classification for Hybrid RAG: A Multi-Domain Question Answering System

Group 7
2229023 Eunna Lee
2391019 Jungyeon Lee
2391022 Seonyeong Jeong

Contents

01 System Architecture

02 ewha.pdf
University Regulations

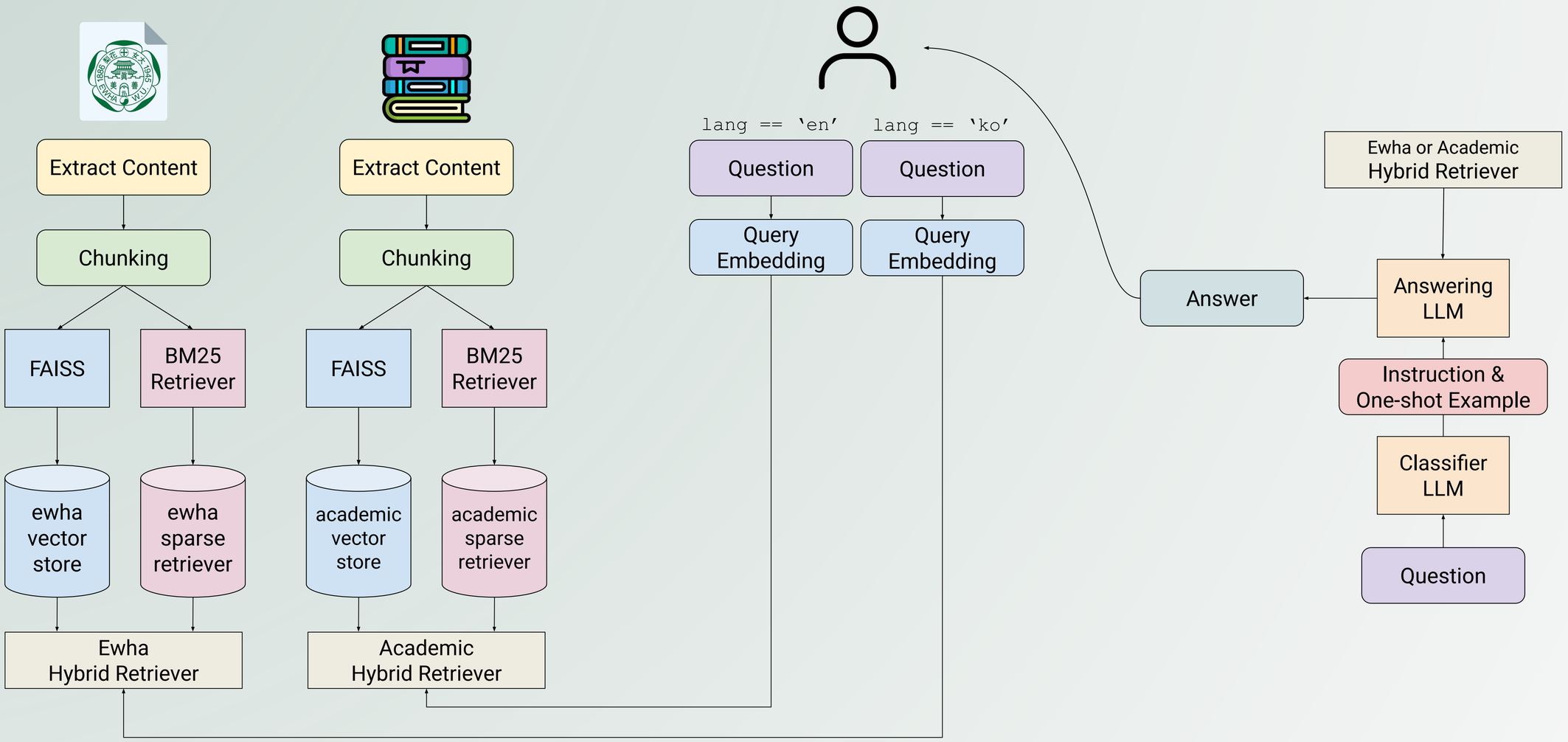
03 MMLU-pro
External Document Collection

04 RAG Knowledge Base

05 RAG

06 Accuracy

System Architecture



ewha.pdf : Content Extraction

Source:

Ewha University Academic Regulation document (PDF)

Problem:

Contains **table** layouts

😕 Standard text extraction cannot recover table information

Solution:

- Used **Upstage Document Parse API** with **ocr='force'**
- Leverages layout analysis to recognize table structure
- Extracts tables as structured markdown
- Manual verification and correction

[별표 2] (개정 2014.11.21., 2015.9.18., 2016.2.16., 2016.3.31., 2017.2.8.)

학사학위의 종류(제50조 관련)

대 학	학위의 종류	학과 또는 전공
인문과학대학	문학사	국어국문학 중어중문학 불어불문학 독어독문학 사학 철학 기독교학 영어영문학
사회과학대학	정치학사 행정학사 경제학사 문학사	정치외교학 행정학 경제학 문헌정보학 사회학 사회복지학 심리학 소비자학
	커뮤니케이션학사	커뮤니케이션·미디어학
자연과학대학	이학사	수학 통계학 물리학 화학·나노과학

[별표 1] (개정 2017.5.4., 2017.5.15.)

학부, 학과, 전공별 입학정원(2019학년도) (제5조 관련)

대학	학부/학과/전공	입학정원
인문과학대학	국어국문학과 중어중문학과 불어불문학과 독어독문학과 사학과 철학과 기독교학과	299
	영어영문학부	91
	소계	390
사회과학대학	정치외교학과 행정학과 경제학과 문헌정보학과 사회학과 사회복지학과 심리학과 소비자학과	290
	커뮤니케이션·미디어학부	79
	소계	369
	수학과 통계학과	118

ewha.pdf : Content Extraction

- Used Upstage Document Parse API with ocr='force'
- Leverages layout analysis to recognize table structure
- Extracts tables as structured markdown
- **Manual verification and correction**

조형예술대학	조형예술학부 동양화전공 서양화전공 조소전공 도자예술전공	113
조형예술대학	디자인학부	80
조형예술대학	패션디자인전공	
	섬유·패션학부 섬유예술전공 소계	47 240

조형예술대학	조형예술학부 동양화전공 서양화전공 조소전공 도자예술전공	113
조형예술대학	디자인학부	80
조형예술대학	섬유·패션학부 섬유예술전공 패션디자인전공	47
조형예술대학	소계	240

등급	성적점
A+	4.3
A°	4.0
A-	3.7
B+	3.3
B°	3.0
B	2.7
C+	2.3
C°	2.0
CT	1.7
D+	1.3
D°	1.0
DT	0.7
F	0

등급	성적점
A+	4.3
A°	4.0
A-	3.7
B+	3.3
B°	3.0
B	2.7
C+	2.3
C°	2.0
C-	1.7
D+	1.3
D°	1.0
D-	0.7
F	0

대 학	학위의 종류	학과 또는 전공
인 문 과 학 대 학	문 학 사	국중불독사철기영 어어어어 국중불독 문문문문 학학학학학학학 독교

대 학	학위의 종류	학과 또는 전공
인문과학대학	문학사	국어국문학
인문과학대학	문학사	중어중문학
인문과학대학	문학사	불어불문학
인문과학대학	문학사	독어독문학
인문과학대학	문학사	사학
인문과학대학	문학사	철학
인문과학대학	문학사	기독교학
인문과학대학	문학사	영어영문학

ewha.pdf : Chunking

Academic regulations follow statute structure (제 NN조)

→ Statute boundaries provide natural semantic units

Prioritize **statute article boundaries (“\n제”)** as primary separator

👍 Maintains regulatory structure

👍 Improves retrieval precision

👍Reduces false splits on complex clauses

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,      # ~1-2 article clauses
    chunk_overlap=100,   # preserve inter-clause context
    separators=[
        "\n제",          # article boundaries
        "\n\n",          # paragraph breaks
        "\n",            # line breaks
        ".",             # sentence endings
        " ",             # word-level split
        ""
    ]
)
```

Challenge: Retrieval Failure on Tabular Data

Problem:

Consistent failure on table-related questions

(e.g., Admission quotas, Degree types)

QUESTION25) 2019학년도 입학 정원에 대한 내용으로 옳은 것을 고르시오.

(A) 휴먼기계바이오공학부 입학 정원과 무용과의 입학 정원의 합은 149명이다.
(B) 무용과의 입학정원은 38명이다.
(C) 수학교육과와 국어 교육과의 입학 정원은 각각 37명과 27명이다.
(D) 음악 대학의 학생수는 자연과학대학의 학생수보다 많다.

Challenge: Retrieval Failure on Tabular Data

QUESTION25) 2019학년도 입학 정원에 대한 내용으로 옳은 것을 고르시오.

(A) 휴먼기계바이오공학부 입학 정원과 무용과의 입학 정원의 합은 149명이다.

(B) 무용과의 입학정원은 38명이다.

(C) 수학교육과와 국어 교육과의 입학 정원은 각각 37명과 27명이다.

(D) 음악 대학의 학생수는 자연과학대학의 학생수보다 많다.

단계 1: [조건부 규칙 추출]
- **기본 규칙**: EWHA.pdf의 입학 정원 데이터는 2019학년도 기준 명시적 언급이 없으나, 제공된 표가 최신 정보로 가정.
- **예외 조건 없음**: 모든 선택지는 표 내 수치 비교로 판단 가능.

단계 2: [선택지별 분석]
(A) 휴먼기계바이오공학부 입학 정원과 무용과의 입학 정원의 합은 149명이다.
- **근거**: 표에는 "휴먼기계바이오공학부" 언급이 없으며, 무용과 정원은 38명. → **불부합**
- **논리적 결함**: 학과명 불일치 및 무용과 정원 단독 확인 불가.

(B) 건반악기과의 입학정원은 164명이다.
- **근거**: 음악대학 전체 정원(건반악기과+관현악과+성악과+작곡과+한국음악과)은 164명. 건반악기과 단독 정원은 "음악학부" 표에서 188명 중 일부. → **불부합**
- **논리적 결함**: 전체 음악대학 정원과 건반악기과 정원 혼동.

(C) 수학교육과와 국어 교육과의 입학 정원은 각각 37명과 27명이다.
- **근거**: EWHA.pdf에 "수학교육과" 및 "국어 교육과" 관련 데이터 없음. → **불부합**
- **논리적 결함**: 근거 자료 부재.

(D) 음악 대학의 학생수는 자연과학대학의 학생수보다 많다.
- **근거**:
  - 음악대학: 음악학부(188명) + 무용과(38명) = **226명**.
  - 자연과학대학: 표에 명시되지 않음(제공된 데이터는 엘텍공과대학, 공과대학, 음악대학만 존재). → **불부합**
- **논리적 결함**: 자연과학대학 정원 미제공으로 비교 불가.

단계 3: [논리적 충돌 검토]
- 모든 선택지는 EWHA.pdf의 **2019학년도 입학 정원**과 직접적/간접적 불일치.
- (A)는 학과명 오류, (B)는 전체 정원 오해석, (C)는 자료 부재, (D)는 비교 대상 미제공.

단계 4: [최종 판단 및 근거]
- **정답 없음**: 제공된 문맥(EWHA.pdf)에서 2019학년도 입학 정원을 직접 확인할 수 있는 선택지 없음.
- 단, (C)의 "수학교육과"와 "국어 교육과"는 표 외부 학과로 추정되며, 다른 선택지는 명백한 오류.

최종 답변: [ANSWER]: (X)
- **사유**: 모든 선택지가 EWHA.pdf의 명시된 데이터와 모순되거나 근거 불충분. 문제 오류 가능성 있음.

Challenge: Retrieval Failure on Tabular Data

Causes of Problem:

Size-Based Fragmentation

- **Large tables exceed chunk_size limit**
- **Rows & columns scattered across multiple chunks**

  (Department names can be separated from quotas.)

  → LLM cannot match entities to data

  → Coherent reasoning impossible

[별표 1] (개정 2017.5.4., 2017.5.15.)

학부, 학과, 전공별 입학정원(2019학년도) (**제5조** 관련)

대학	학부/학과/전공	입학정원
인문과학대학	국어국문학과 중어중문학과 불어불문학과 독어독문학과 사학과 철학과 기독교학과	299
인문과학대학	영어영문학부	91
인문과학대학	소계	390
사회과학대학	정치외교학과 행정학과 경제학과 문헌정보학과 사회학과 사회복지학과 심리학과 소비자학과	290
사회과학대학	커뮤니케이션·미디어학부	79
사회과학대학	소계	369
자연과학대학	수학과 통계학과 물리학과	118

...

Solution:

Semantic Table Parsing & Normalization

- Dedicated pipeline for structured data extraction
  - Extract tables as complete units
  - Normalize each row: Year + College + Department + Quota + Degree
  - Preserve full context for retrieval

Semantic Table Parsing

ewha_parsed.md

Source:
OCR-extracted markdown with embedded table markers

Format:
Multiple [별표 1] tables, [별표 2], [별표 3], ...

Challenge:
- **Multiple [별표 1] tables for different academic years**
- Admission quota tables from 2014-2019 all labeled as [별표 1]
- No year metadata in marker itself → Need intelligent parsing
- **Risk of conflicting versions ([별표 2] appears twice)**

1. Pattern Recognition & Extraction

Method: Regex-based [별표 N] pattern detection with context analysis

```
table_pattern = r'\[별표
(\d+)\](.*?)(?=\n\[별표|\Z)'
matches = list(re.finditer(table_pattern,
md_content, re.DOTALL))

# Extract each [별표 N] block and
surrounding content
for match in matches:
    table_num = match.group(1)
    block_text = match.group(2).strip()
```

Output:
- **Identify all [별표 N] markers**
- **Extract complete table blocks with headers**
- Preserve surrounding metadata (year references)

2. Year-Based Table Classification

Discover that multiple [별표 1] tables exist.

Key Insight:
Year information is in table CONTEXT, not in [별표 N] marker
→ Parse headers and surrounding text to **extract year metadata**

3. Disambiguation & Versioning Control

[별표 2] (현행) : Active ✓
[별표 2] (경과조치) : Outdated ✗

Filtering Logic:

```
if table_num == '2' and
'(경과조치' in block_text:
    continue
```

✓ [별표 1] 2019학년도 테이블: 49행 x 3열
✓ [별표 1] 2018학년도 테이블: 49행 x 3열
✓ [별표 1] 2017학년도 테이블: 49행 x 3열
✓ [별표 1] 2016학년도 테이블: 66행 x 4열
✓ [별표 1] 2015학년도 테이블: 50행 x 4열
✓ [별표 1] 2014학년도 테이블: 46행 x 4열
✓ [별표 2] 학년도 테이블: 110행 x 3열
✓ [별표 3] 학년도 테이블: 1행 x 6열
총 8개 테이블 추출

Semantic Table Parsing

Table Normalization: **Dynamic Column Mapping**

Challenge: Tables have inconsistent headers & column orders

↓

Solution: Dynamic semantic column detection

↓

Result: All rows normalized to unified schema

```
cols = {'college': -1, 'dept': -1, 'quota': -1, 'degree': -1}

for i, h in enumerate(headers):
    h_clean = h.replace(" ", "")
    if '대학' in h_clean and '학위' not in h_clean: cols['college'] = i
    elif '학부' in h_clean or '학과' in h_clean or '전공' in h_clean: cols['dept'] = i
    elif '정원' in h_clean: cols['quota'] = i
    elif '학위' in h_clean or '종류' in h_clean: cols['degree'] = i
```

```
if '소계' in dept_text or '합계' in dept_text:
    metadata_list.append({
        'type': 'subtotal',
        'year': year,
        'college': college,
        'label': dept_text,
        'quota': quota,
        'degree': degree,
    })
    continue

metadata_list.append({
    'type': 'general_info', # Unified metadata type
    'year': year,
    'college': college,
    'department': dept_text,
    'quota': quota,
    'degree': degree, # Store degree info
    'table_id': table_id,
    'row_index': row_idx
})
```

Table Reconstruction

Convert to LLM-Friendly Natural Language

(e.g., “사범대학 국어교육과: 입학정원 27명”)

- Direct semantic information for LLM
- Unified representation across all extracted tables

Post-Processing:

- Remove original tables to prevent duplication

Chunking Strategy

Articles:

- Meaning-based chunking
- Typically one chunk = one article

Natural Language Tables:

- Max 40 rows per chunk
  (Prevents exceeding Upstage API token limits)

Semantic Table Parsing

Validation: Retrieval Quality Check

🔎 Retrieval Test 1: Query **'문학사'**

[청크 샘플]
[표 2 정보 (연도: None, Part 1)]

- 인문과학대학 국어국문학: 수여학위 **'문학사'**
- 인문과학대학 중어중문학: 수여학위 **'문학사'**
- 인문과학대학 불어불문학: 수여학위 **'문학사'**
- 인문과학대학 독어독문학: 수여학위 **'문학사'**
- 인문과학대학 사학: 수여학위 **'문학사'**
- 인문과학대학 철학: 수여학위 **'문학사'**

🔎 Retrieval Test 2: Query **'수학교육과'**

[Source: regulation, Year: None]
행당시 생물학과, 교육학과의 교육학전공, 유아교육전공, 초등교육
전공 및 과학교육과의 수학전공 재적생은 이 학칙에 의하여 각각 생물과학과, 교육학과, 유아
교육과, 초등교육과 및 **수학교육과**의 재적생으로 본다.

부칙(학무25210-508, 1992. 3. 16 개정)
이 학칙은 1992년 3월 16일부터 시행한다.
...

[Source: table_full, Year: 2019]
지리교육전공: 입학정원 71명
- 사범대학 국어교육과: 입학정원 27명
- 사범대학 과학교육과 물리교육전공 화학교육전공 생물교육전공 지구과학교육전공: 입학정원 86명
- 사범대학 **수학교육과**: 입학정원 27명
- **사범대학 소계: 입학정원 408명**
- 경영대학 경영학부: 입학정원 116명
- **경영대학 소계: 입학정원 116명**
...

[Source: table_full, Year: 2018]
지리교육전공: 입학정원 71명
- 사범대학 국어교육과: 입학정원 27명
- 사범대학 과학교육과 물리교육전공 화학교육전공 생물교육전공 지구과학교육전공: 입학정원 86명
- 사범대학 **수학교육과**: 입학정원 27명
- **사범대학 소계: 입학정원 408명**
- 경영대학 경영학부: 입학정원 116명
- **경영대학 소계: 입학정원 116명**
...

External Document Collection

Category-Based Collection via Wikipedia API

Philosophy	Psychology	Business	Law	History
"Epistemology"	"Cognitive psychology"	"Business strategy"	"Contract law"	"Ancient history"
"Metaphysics"	"Behavioral psychology"	"Marketing management"	"Tort law"	"Medieval history"
"Ethics"	"Developmental psychology"	"Organizational behavior”	"Constitutional law"	"Modern history"
"Logic"	"Social psychology"	"Financial management"	"Criminal law"	"World War I"
"Aesthetics"	"Clinical psychology"	"Accounting"	"Property law"	"World War II"
"Philosophy of mind"	"Abnormal psychology"	"Strategic planning"	"Administrative law"	"Cold War"
"Philosophy of language"	"Learning theory"	"Management theory"	"International law"	"Industrial Revolution"
"Political philosophy"	"Memory"	"Organizational structure"	"Labor law"	"American history"
"Philosophy of science"	"Cognition"	"Human resources"	"Environmental law"	"European history"
"Ontology"	"Motivation"	"Business ethics"	"Family law"	"Asian history"
"Phenomenology"	"Emotion"	"Supply chain"	"Corporate law"	"African history"
"Existentialism"	"Personality theory"	"Entrepreneurship"	"Evidence law"	"Cultural history"
"Ancient philosophy"	"Group dynamics"	"Organizational culture"	"Civil procedure"	"Intellectual history"
"Modern philosophy"	"Psychopathology"	"Decision-making processes"	"Jurisprudence"	"Economic history"
"Virtue ethics"				
→ 15	→ 14	→ 14	→ 14	→ 14

External Document Collection

[Philosophy]

Source: **Stanford Encyclopedia of Philosophy**

Method: Automated **Web Crawling**

- HTTP requests + BeautifulSoup HTML parsing
- DOM selector: div#main-text (content extraction)
- **Hierarchical segmentation by h2/h3 headings (Preserves topic coherence across sections)**
- Batch multi-page crawling via loop iteration

Output: 412 Semantically Segmented Sections

Each section = one heading boundary
Maintains hierarchical organization (essay structure)

```
stanford_sep_urls = {
    "Epistemology": "https://plato.stanford.edu/entries/epistemology/",
    "Metaphysics": "https://plato.stanford.edu/entries/metaphysics/",

    "Virtue Ethics": "https://plato.stanford.edu/entries/ethics-virtue/",
    "Deontological Ethics":
"https://plato.stanford.edu/entries/ethics-deontological/",

    "Classical Logic":
"https://plato.stanford.edu/entries/logic-classical/",
    "Propositional Logic":
"https://plato.stanford.edu/entries/logic-propositional/",

(...)

    "Artificial Intelligence":
"https://plato.stanford.edu/entries/artificial-intelligence/",
    "Personal Identity":
"https://plato.stanford.edu/entries/identity-personal/",
    "Moral Relativism":
"https://plato.stanford.edu/entries/moral-relativism/",
    "Skepticism": "https://plato.stanford.edu/entries/skepticism/",
    "Logical Consequence":
"https://plato.stanford.edu/entries/logical-consequence/",
}
```

External Document Collection

PDF-to-Document Conversion

[Philosophy]

Source Materials:

- “Introduction to Philosophy”
- “Introduction to Philosophy: Ethics”

Tool:

`UpstageDocumentParseLoader(ocr=False)`

[LAW]

Source Materials:

- “Business Law I Essentials”
- “Criminal Law (Lisa Storm)”
- “Law School Essentials-Constitutional Law”
- “Exercises in Civil Procedure”
- “Tort Law Cases and Commentaries”

External Document Collection

PDF-to-Document Conversion

[Psychology]

Source Materials:

- “Psychology 2e”
- “Ethical Principles of Psychologists and Code of Conduct“

[History]

Source Materials:

- “Modern World History New Perspectives”
- “World History-Cultures, States, Societies to 1500”
- “The History of Our Tribe-Hominini”

[Business]

Source Materials:

- “Introduction To Business”

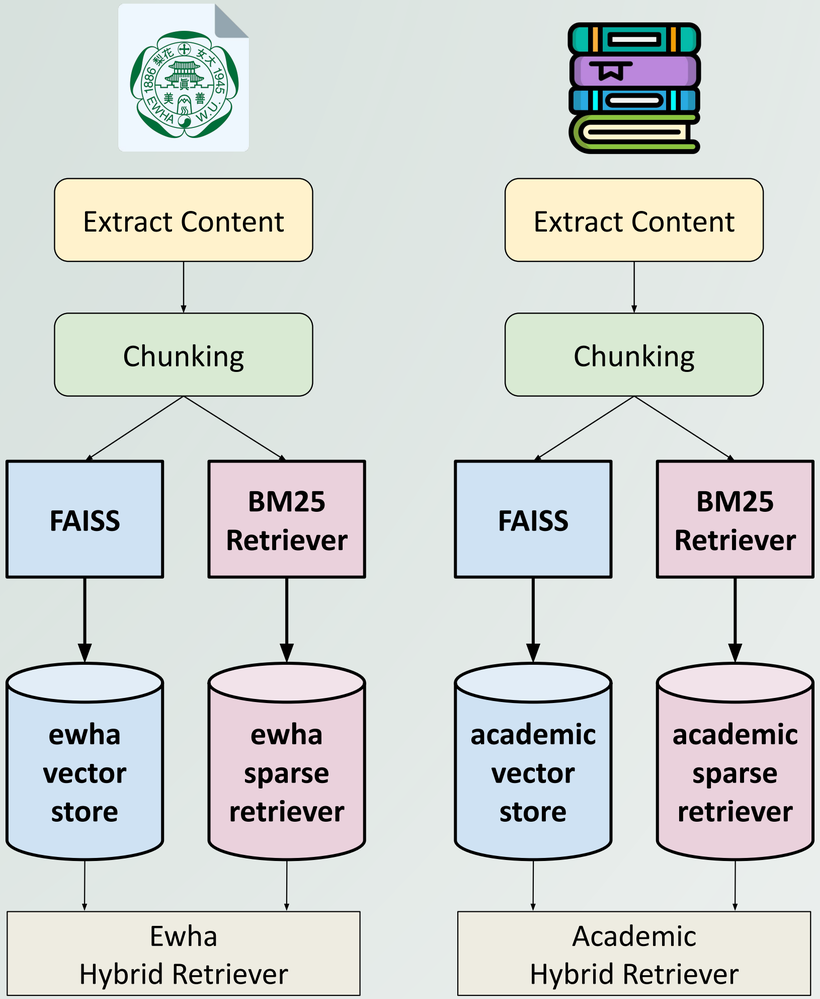
External Documents : Chunking Strategy

Adaptive Chunking by Document Source

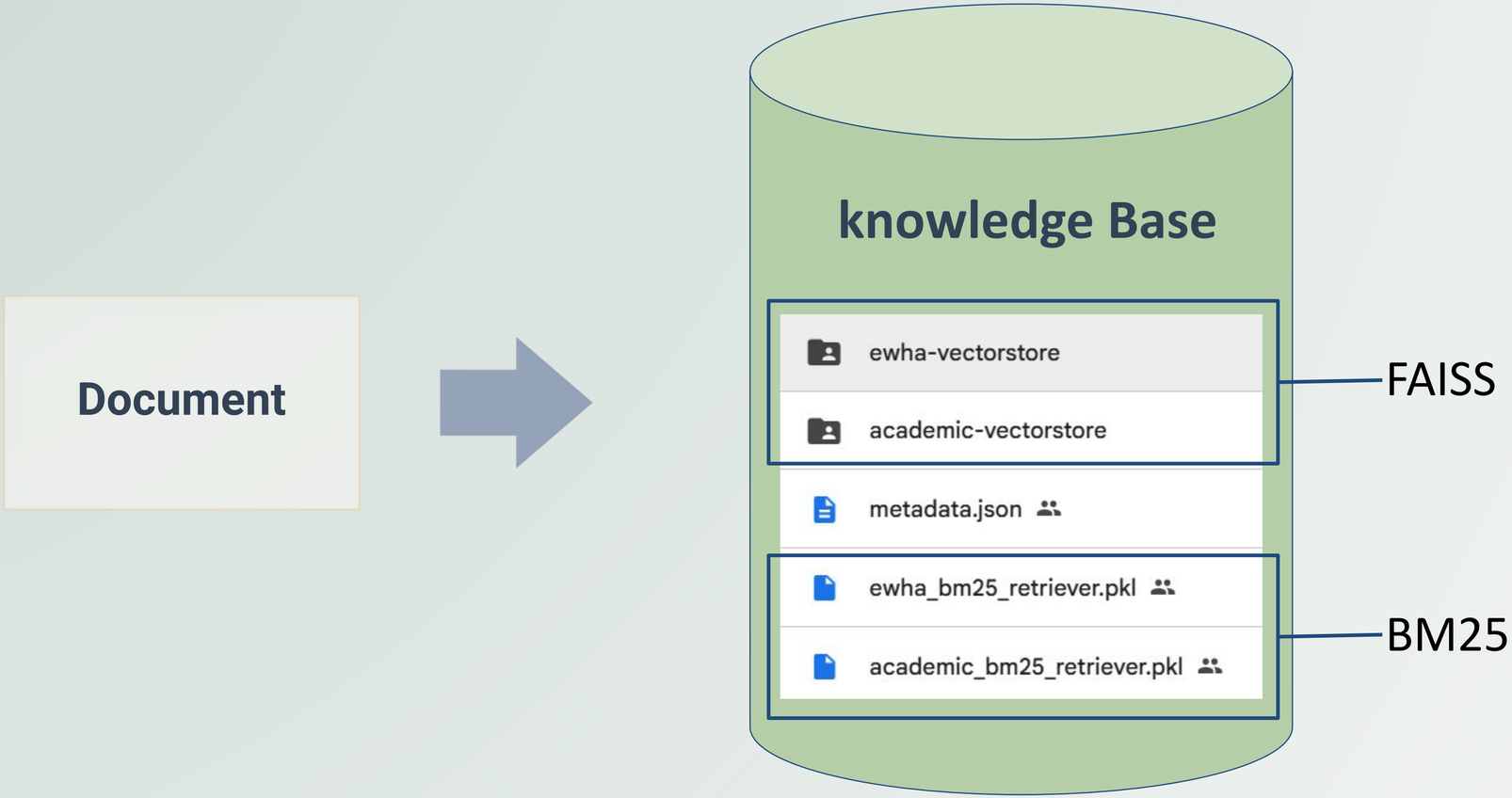
Method:
`RecursiveCharacterTextSplitter`

	Wikipedia Documents	Stanford Encyclopedia of Philosophy (SEP)	PDF - Academic Textbooks
Chunk size / Overlap	1024 / 150	1500 / 200	1200 / 120
Separators	Double newline, single newline, period	Longer chunks preserve hierarchical structure & semantic	Default
Rationale	Moderate chunk size balances coherence & retrieval granularity	Longer chunks preserve hierarchical structure & semantic continuity of philosophical essays	Fixed-size segments for uniform processing

RAG Knowledge Base Structure



RAG Knowledge Base Structure



RAG Knowledge Base 1) FAISS

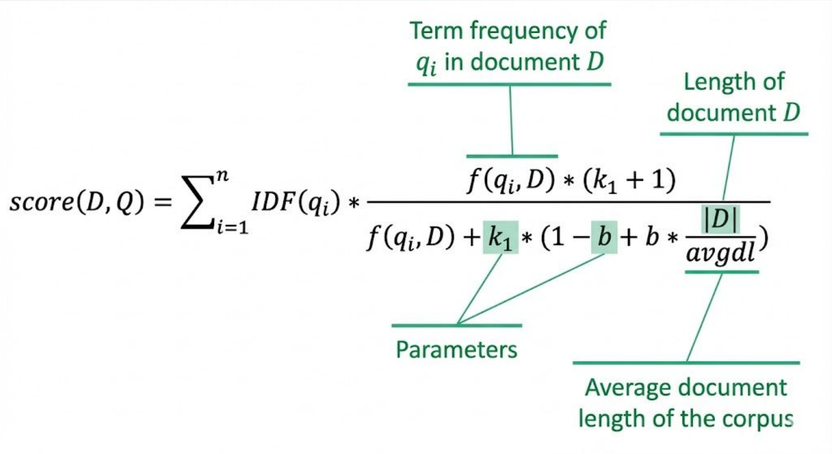
1. **Embedding Model:** **Upstage solar-embedding-1-large**
   - **Specialty:** Bilingual (KR/EN) processing & Long-Context understanding
   - **Role:** Transforms text into high-dimensional vector space.

```
upstage_embeddings = UpstageEmbeddings(
    api_key=UPSTAGE_API_KEY,
    model="solar-embedding-1-large"
)
```

2. **Data Representation:** **Dense Vector**
   - **Key Feature:** Captures **Semantic Context** (Synonyms, Nuances)
   - **Benefit:** Enables meaning-based search beyond exact keywords
     - *e.g., "Rule Violation" -> Retrieves "Disciplinary Action"*

3. **Metric:** **Cosine Similarity**
   - **Reason:** The standard standard for measuring semantic similarity in NLP

RAG Knowledge Base 2) BM25



1. Retrieval-Based Approach

- Not Embedding: No vector space transformation process
- Exact Matching: Based on direct keyword overlap between query and document
- Role: Traditional Information Retrieval (IR) technique, not Deep Learning

2. Deterministic Algorithm

- Statistical Logic: Uses fixed formulas based on **Term Frequency (TF) & Document Length**
- No Trainable Parameters: No weight learning required
- Fixed Output: Consistent results for the same input (Deterministic)

3. Efficiency & Independence

- No Pre-training: Zero training cost, ready to use immediately
- Independent: Compatible with any embedding model (e.g., Solar, OpenAI) for easy Ensembling

RAG Knowledge Base 3) meta.json


```
{
  "ewha": {
      "vectorstore_path": ".../ewha-vectorstore",
      "bm25_path": ".../ewha_bm25_retriever.pkl",
      "num_chunks": 41
  },
  "academic": {
      "vectorstore_path": ".../academic-vectorstore",
      "bm25_path": ".../academic_bm25_retriever.pkl",
      "num_chunks": 25191
  }
}
```

RAG Knowledge Base Merge

Jungyeon
(Ewha, Wikipedia, Philosophy)

Seonyeong
(Psychology, Business)

Eunna
(Law, History)

Merge Logic (FAISS & BM25)

Final knowledge Base

- ewha-vectorstore
- academic-vectorstore
- metadata.json
- ewha_bm25_retriever.pkl
- academic_bm25_retriever.pkl

FAISS

BM25

RAG Knowledge Base Merge 1) FAISS

- Extending Dense Vector Store by 'Merging' External Dense Vector Stores

```
# 1. Load the Base Store
main_store = FAISS.load_local(MY_STORE_PATH, ...)

# 2. Iterate & Merge Teammate Stores
for path in TEAMMATE_PATHS:
    other_store = FAISS.load_local(path, ...)
    main_store.merge_from(other_store)  # 핵심 기능

# 3. Save Final Store
main_store.save_local(FINAL_STORE_PATH)
```

merge_from() : Efficiently combines physically separated indexes into a single index without additional embeddings

RAG Knowledge Base Merge 2) BM25

- Re-building Retriever by Extracting All Documents

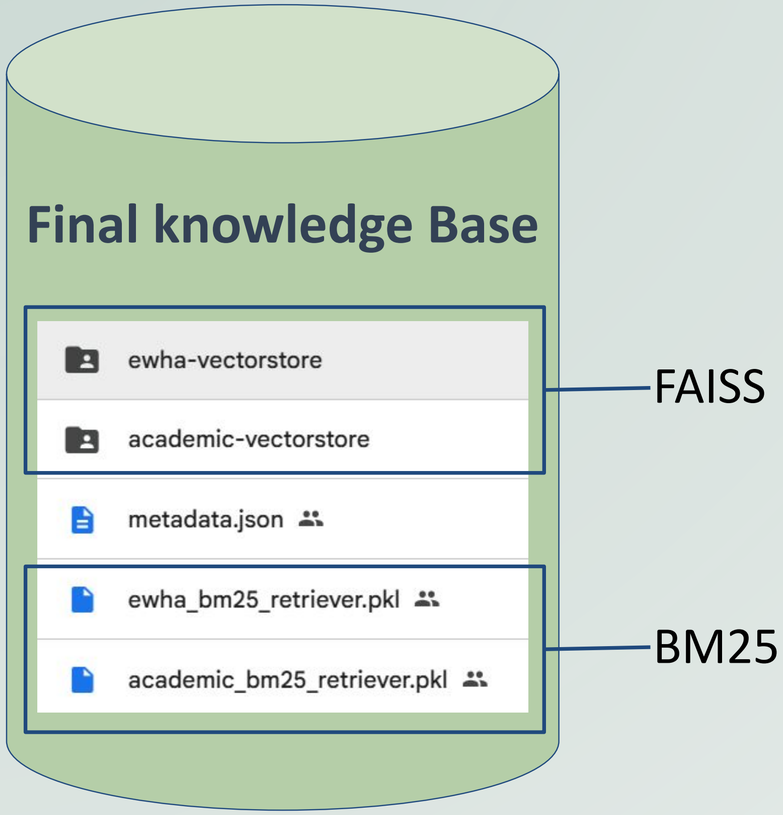
1. Load Local BM25 File
2. Extract internal .docs attribute
3. Load Teammate's BM25 & Extract .docs
4. Execute BM25Retriever.from_documents(all_docs)
5. Save merged object via pickle

```
all_docs = []
# 1. Document Extraction
all_docs.extend(my_retriever.docs)
all_docs.extend(tm_retriever.docs)

# Re-build Index (Recalculate IDF)
combined_bm25 =
  BM25Retriever.from_documents(all_docs)

# Save Object
pickle.dump(combined_bm25, f)
```

RAG : Hybrid Retriever



Relevant Context

Use Hybrid Retriever

1. FAISS (60%)

   EWHA: k=3 / Academic: k=3

2. BM25 (40%)

RAG : LLM

- **Upstage Model**
  - Stage 1

```
llm_classifier = ChatUpstage(api_key=UPSTAGE_API_KEY, model="solar-pro2")
```

  - Stage 2

```
llm_reasoning = ChatUpstage(api_key=UPSTAGE_API_KEY, model="solar-pro2")
```

- **LLM Usage - chain**
  - Stage 1

```
classifier_chain = prompt | llm_classifier | StrOutputParser()
raw_out = classifier_chain.invoke({"question": question})
```

  - Stage 2

```
rag_chain = prompt_template | llm_reasoning | StrOutputParser()
```

RAG : LLM

solar-pro2

Parameter: 30.9B

Context length: 65,536

Reasoning mode X

	Solar Pro 2 Advanced reasoning, tool use, and multilingual support Learn more Playground	Solar Mini A lightweight model providing high-quality language processing Learn more Playground
Intelligence	●●●●●	●●
Speed	⚡⚡⚡⚡	⚡⚡⚡⚡
Reasoning mode	✓	⊗
PRICING PER TIER	PER 1M TOKENS	PER 1M TOKENS
Input Token	$0.15	$0.15
Output Token	$0.6	$0.15
Cached Token	$0.015	-
CONTEXT		
Context length	65,536	32,768
Parameters	31B	10.7B
Training data cutoff	Mar, 2025	Sep, 2024
CAPABILITIES		
Chat	✓	✓
Structured outputs	✓	✓
Function calling	✓	✓
Reasoning	✓	⊗

RAG : LLM

solar-pro2

Parameter: 30.9B

Context length: 65,536

Reasoning mode X

High performance on MMLU & Korean tasks without reasoning

			upstage		Alibaba.com				deepseek		OpenAI	
Model Name			Solar Pro 2 [R, Reasoning]	Solar Pro 2	Qwen3-32B [R]	Qwen3-32B	qwen3-235b -a22b [R]	qwen3-235b -a22b	Deepseek R1-0528 [R]*	Deepseek-v3-0 324*	o3-mini (medium)[R]	gpt-4o -2024-11-20
Model Size			30.9B	30.9B	32B	32B	235B-A22B	235B-A22B	671B-A37B	671B-A37B	N/A	N/A
KOR	General	Ko-Arena-Hard-v0.1 Judge: GPT-4.1-2025-04-14	71.09%	61.90%	50.55%	51.40%	53.20%	70.72%	77.88%	58.65%	60.07%	41.85%
		Ko-MMLU 0shot, CoT	84.64%	79.96%	81.52%	75.24%	82.40%	79.56%	86.24%	79.56%	81.64%	84.08%
		KMMLU 0shot	77.04%	72.04%	73.88%	68.44%	75.04%	71.92%	79.28%	71.92%	72.36%	73.08%
		Hae-Rae 1.0	86.22%	86.22%	74.25%	71.59%	77.96%	78.80%	81.73%	84.59%	77.44%	85.57%
		Ko-IFEval	86.52%	86.14%	84.59%	83.06%	88.51%	66.53%	88.11%	90.67%	92.12%	83.56%
		Ko-MT-Bench	88.57%	89.06%	85.16%	86.70%	85.77%	87.82%	85.77%	84.88%	89.95%	88.15%
	Domain	KBL law	75.51%	69.43%	63.16%	64.17%	67.31%	68.62%	70.04%	55.57%	67.11%	74.09%
		KBankMMLU** Finance	71.04%	65.20%	64.28%	56.12%	63.72%	58.44%	73.28%	61.36%	61.92%	61.96%
		KorMedMCQA Medical	87.40%	83.00%	83.12%	73.92%	84.92%	81.20%	88.72%	81.60%	85.56%	87.36%
ENG		MMLU 0shot, CoT	88.16%	84.92%	86.44%	81.36%	89.00%	82.00%	90.60%	86.88%	85.76%	86.16%
		MMLU-Pro 5shot, CoT	76.52%	72.48%	76.88%	68.84%	81.08%	72.24%	83.44%	72.69%	78.12%	73.28%
		IFEval	87.82%	87.81%	87.54%	87.76%	89.96%	71.37%	89.55%	89.45%	94.32%	85.52%
		MT-Bench	88.50%	88.56%	87.69%	86.94%	89.06%	87.13%	87.25%	88.19%	87.25%	89.25%
JPN		Ja-MMLU 0shot, CoT	85.08%	79.80%	78.95%	76.28%	83.32%	79.48%	86.56%	82.36%	83.44%	82.40%
		Ja-IFEval	82.87%	81.11%	83.48%	80.18%	84.60%	61.80%	82.46%	83.74%	87.37%	78.24%

* DeepSeek 모델 성능은 공식 API를 통해 측정되었습니다.
** 업스테이지가 국내 주요 금융사와 협력하여 개발한 비공개 데이터셋 기반 벤치마크로, 한국 금융 도메인에 특화된 언어 모델의 이해 및 추론 능력을 평가합니다.

LLM-Based Category Classification: Motivation

Problem: The Limitations of Generic Prompting

Using a single, generic prompt for questions across diverse academic domains
→ Low accuracy on MMLU tasks
→ Attempting category-specific prompts
→ **Explicit classification required!**

Classification based on question keywords
⚠️ **reliability issues**
(Failed to classify correctly)

```python
def classify_question_type(question):
    """문제 유형 분류"""
    language = detect_language(question)

    if language == "korean":
        return 4, "ewha_regulation", "korean"
    else:
        # 영어 문제인 경우, 키워드로 카테고리를 추정하여 힌트를 활성화합니다.
        q_lower = question.lower()
        if any(x in q_lower for x in ['law', 'legal', 'court', 'crime', 'justice', 'statute']):
            category = "law"
        elif any(x in q_lower for x in ['psycholog', 'behavior', 'cognitive', 'freud', 'mental']):
            category = "psychology"
        elif any(x in q_lower for x in ['business', 'market', 'econom', 'finance', 'manage']):
            category = "business"
        elif any(x in q_lower for x in ['history', 'century', 'war', 'empire', 'revolution']):
            category = "history"
        elif any(x in q_lower for x in ['philosoph', 'ethics', 'moral', 'kant', 'plato']):
            category = "philosophy"
        else:
            category = "academic_subject" # fallback

        return 10, category, "english"
```

LLM-Based Category Classification

Brainstorming

Leveraging Unrestricted LLM Calls
→ **Why not let the LLM classify?** 🤔

1. **LLM reads question only → Classify category**
2. **Generate answer based on the classified category**

Conceptualization

"2-Stage Strategy"
Utilize LLM in two phases: **Category Classification** and **Problem Solving (Answering)**

"LLM-based Routing"
- LLM itself serves as the **Router**
- Intrinsic knowledge determines the most appropriate RAG **routing path** for the query

LLM-Based Category Classification: Methodology

Predict two categories (Primary & Secondary) → Provide Primary category & One-shot example explicitly

- **Result/Finding**
  Requiring two predictions seemed to alleviate the classification burden on the LLM, leading to a significant boost in Primary Category classification accuracy.

- **Implementation**
  While two categories were predicted, the Secondary category was intentionally unused in RAG/Answering phase.

```
"""You are an expert at answering difficult multiple-choice questions (MMLU-Pro style), specializing in
**{primary_cat.upper()}**.

Your job:
- Use the **academic context** below as your PRIMARY source.
- BUT you may also use your own background knowledge **to fill in gaps or break ties**.
- If the context gives a **clear definition, rule, or condition** related to ** {primary_cat.upper()}**,
  you MUST follow that strictly, even if it conflicts with your memory.

Question: {{question}}

Academic Context: {{context}}

[Examples]
{few_shot_text}

Guidelines:
1. First, extract the key definitions, rules, or facts from the context that specifically relate to ** {primary_cat.upper()}**.
2. Then check how each option {answer_options} matches or conflicts with those facts.
3. If the context is vague or incomplete, rely ONLY on standard **textbook knowledge of {primary_cat.upper()}** to choose. Do
not mix concepts from other disciplines.
4. Prefer the option that:
   - Best matches the core **{primary_cat.upper()}** principle in the context, AND
   - Sounds like a typical textbook or exam answer.
5. Avoid speculative or overly creative interpretations.

Format your response exactly as follows:

Step 1: [Key concepts and relevant information]
Step 2: [Understanding the question's requirements]
Step 3: [Option analysis]
Final Answer: [ANSWER]: (X) where X is one of {answer_options}"""
```

▸ RAG prompt template for multi-cat

Execution Flow

1. **Knowledge Base Initialization**
   - Ewha Hybrid Retriever
   - Academic Hybrid Retriever
2. **Component Setup**
   - Category Classification Chain (w/ Prompt)
   - Classification Output Parser
   - Category-Specific One-Shot Examples
   - One-Shot Prompt Formatter
   - Problem-Solving Prompt Templates
   - Answer Extraction Logic
3. **Data Preparation**
4. **[Stage 1: Classification]** Execute Category Classification Chain
5. **[Stage 2: Problem Solving]** Execute RAG Chain for Problem Solving
6. **Results & Analysis**
   - Save Execution Results
   - Print Accuracy
   - Conduct Detailed Result Analysis

Execution Flow

1. **Knowledge Base Initialization**
   - Ewha Hybrid Retriever
   - Academic Hybrid Retriever
2. **Component Setup**
   - Category Classification Chain (w/ Prompt)
   - Classification Output Parser
   - Category-Specific One-Shot Examples
   - One-Shot Prompt Formatter
   - Problem-Solving Prompt Templates
   - Answer Extraction Logic
3. **Data Preparation**
4. **[Stage 1: Classification]** Execute Category Classification Chain
5. **[Stage 2: Problem Solving]** Execute RAG Chain for Problem Solving
6. **Results & Analysis**
   - Save Execution Results
   - Print Accuracy
   - Conduct Detailed Result Analysis

Category Classifier Chain (w/ Prompt)


```
def get_category_classifier_chain_multilabel():
    template = """Analyze the following question and identify the top 2 most relevant
academic categories.

Valid Categories: {categories}

[Instructions]
1. **Primary Category**: The main discipline the question belongs to.
2. **Secondary Category**: A related discipline that might provide useful context or
reasoning rules (e.g., A Law question might have Ethical/Philosophy aspects).
3. If the question purely belongs to one domain, set Secondary to 'None'.
4. Output Format must be EXACTLY:
   PRIMARY: [category_name]
   SECONDARY: [category_name]

Question: {question}

Analysis:"""

    prompt = PromptTemplate(
        template=template,
        input_variables=["question"],
        partial_variables={"categories": ", ".join(VALID_CATEGORIES)}
    )

    return prompt | llm_classifier | StrOutputParser()
```

Among the 5 Categories,
the LLM was instructed to:

- Select the **main discipline** the question belongs to → **Primary Category**
- Select a **related or potentially useful discipline** → **Secondary Category**

→ **Minimizes the classification burden**

→ **Achieves high accuracy for the Primary Category**

Classification Output Parser


```python
VALID_CATEGORIES = ['business', 'history', 'law', 'philosophy', 'psychology']

def parse_multilabel_output(text):
    text = text.lower()

    primary = "general"
    secondary = None

    p_match = re.search(r"primary\s*:\s*([a-z_]+)", text, re.IGNORECASE)
    s_match = re.search(r"secondary\s*:\s*([a-z_]+|none)", text, re.IGNORECASE)

    if p_match:
        val = p_match.group(1).lower()
        if val in VALID_CATEGORIES:
            primary = val
    if s_match:
        val = s_match.group(1).lower()
        if val in VALID_CATEGORIES:
            secondary = val
        elif val == 'none':
            secondary = None
    if "ewha" in text or re.search(r'[가-힣]', text):
        return "korean_regulation", None

    return primary, secondary
```

- **Classification Prompt**
  X enforce strong reasoning

  → **Simple Output Parser**

Category-Specific One-Shot Examples


```
FEW_SHOT_DATABASE = {
    "korean_regulation": [
        {
            "question": "휴학기간은 통산하여 a년을 초과할 수 없는데, 예외적으로 건축학전공은 b년을 초과할 수 없다. a와 b가 상수일 때 a+b의 값을 구하면?",
            "context": "제26조 ④ 휴학기간은 통산하여 3년(건축학전공의 경우 4년, 의예과의 경우 3학기)을 초과할 수 없다.",
            "reasoning": "제26조 4항에 휴학기간은 통산하여 3년을 초과할 수 없다고 명시되어 있으므로 a=3이고, 건축학전공은 4년을 초과할 수 없으므로 b=4이다. 따라서 a+b=7이다.",
            "answer": "(D)"
        }
    ],
    "business": [
        {
            "question": "If the price elasticity of demand for a product is greater than 1 (elastic), which outcome is most likely when the firm lowers price?",
            "context": (
                "Price elasticity of demand measures the percentage change in quantity demanded divided by the percentage change in price. "
                "If elasticity > 1, quantity demanded responds proportionally more than price. Total revenue equals price times quantity. "
                "When demand is elastic, a decrease in price leads to a proportionally larger increase in quantity demanded, raising total revenue."
            ),
            "reasoning": (
                "Apply the definition: elasticity > 1 implies quantity change > price change in percent terms. "
                "Since TR = P * Q, a price cut increases Q enough that TR increases. "
                "Therefore the option stating total revenue will increase is correct."
            ),
            "answer": "[ANSWER]: (A)"
        }
    ],
```

One-Shot Prompt Formatter


```python
PROMPT_TEMPLATES = {
    "en": {
        "header": "[Example - {category}]",
        "question": "Question",
        "context": "Context",
        "reasoning": "Reasoning",
        "answer": "Answer"
    },
    "ko": {
        "header": "[예시]",
        "question": "질문",
        "context": "문맥",
        "reasoning": "추론",
        "answer": "정답"
    }
}
```

```python
def get_few_shot_text(category: str, lang: str = "en"):
    example = FEW_SHOT_DATABASE.get(category,[])
    labels = PROMPT_TEMPLATES.get(lang, PROMPT_TEMPLATES["en"])
    formatted_text = []

    if not example:
        return ""

    for ex in example:
        block = (
            f"\n{labels['header'].format(category=category.upper())}\n"
            f"{labels['question']}: {ex.get('question', '')}\n"
            f"{labels['context']}: {ex.get('context', '')}\n"
            f"{labels['reasoning']}: {ex.get('reasoning', '')}\n"
            f"{labels['answer']}: {ex.get('answer', '')}\n"
        )
        formatted_text.append(block)

    return "\n".join(formatted_text)
```

Problem-Solving Prompt Templates: Korean (1)


```
def get_focused_rag_prompt(primary_cat: str, secondary_cat: str, num_choices: int):
    answer_options = "(A, B, C, D)" if num_choices == 4 else "(A, B, C, D, E, F, G, H, I, J)"
    if primary_cat == "korean_regulation":
        few_shot_text = get_few_shot_text("korean_regulation", lang="ko")
        return PromptTemplate.from_template(f"""당신은 객관식 문제 답변 전문가입니다.
이 문제는 **이화여자대학교 학칙(EWHA.pdf)**을 바탕으로 출제된 문제입니다.

[절대 준수할 규칙]

1. **반드시 EWHA.pdf에 있는 문장과 숫자만** 사용해서 판단합니다.
  - 다른 대학의 학칙, 일반 상식, 법학/교육학 지식 등은 절대 사용하지 마세요.
  - 애매할 때도 추측하지 말고, **EWHA.pdf 문장과 표의 숫자**를 최우선으로 따르세요.
  - 당신의 배경지식보다, EWHA.pdf 문맥과 시험에서 일반적으로 사용하는 정의를 우선하세요.

2. 선택지를 고를 때는 다음 순서를 따릅니다.
  1) 먼저 EWHA.pdf에서 **문제와 직접 연결되는 조항(조문 번호, 문장, 표의 숫자 등)**을 정리합니다.
  2) 각 선택지가 **그 조문을 얼마나 정확히 반영하는지** 비교합니다.
    - 문구, 숫자, 조건(예외 조항 포함)이 조문과 가장 비슷한 것을 우선합니다.
  3) 여러 선택지가 가능해 보이면, **조문과 정반대이거나, 조문에 없는 내용을 가정하는 선택지부터 오답으로** 처리합니다.

3. 문제에 "해당하지 않는 것", "제외되는 것", "제적되지 않는 경우"처럼
  **부정형 표현**이 있을 때는 반드시 아래 순서를 따릅니다.
  - (1) 먼저 EWHA.pdf 문맥에서 **해당/제적/인정 사유** 전체를 정리합니다.
  - (2) 그 목록에 **포함되는 선택지**를 먼저 표시합니다.
  - (3) 그 목록에 **포함되지 않거나, 예외 규정에 해당하는 선택지**를 답으로 선택합니다.

4. 숫자·연한·학점 관련 문제의 경우:
  - 문제에서 묻는 조건(예: "졸업 최소 기준", "조기졸업 기준", "통산 휴학 가능 연한")에
    **가장 정확히 대응하는 숫자**를 고릅니다.
  - 상식적 추론을 하지 말고, **조문에 적힌 그대로** 사용합니다.
```

- **Role**

  Multiple-Choice QA Specialist

- **[Rule 1] Hallucination Prevention**

  Exclusive reliance on explicit context (ewha.pdf)
  Strict prohibition of external knowledge & speculation

- **[Rule 2] Algorithmic Reasoning**

  Mandated workflow:
  Clause Extraction → Option Comparison → Elimination

- **[Rule 3,4] Defensive Prompting**

  Specialized protocols for high-error areas (**Negation** & **Numerals**)

Problem-Solving Prompt Templates: Korean (2)


```
문제: {{question}}
EWHA.pdf 문맥: {{context}}

{few_shot_text}

다음 형식과 순서를 꼭 지켜서 생각하고 답변하세요.

단계 1: [EWHA.pdf에서 발견한 관련 정보]
- 문제와 직접적으로 연결되는 조항(조문 번호, 문장, 표의 숫자 등)을 요약합니다.
- 부정형 문제라면, 먼저 학칙에서 규정한 "해당/제적/인정" 조건 전체를 정리합니다.

단계 2: [추론 과정]
- 위 조항을 바탕으로, 문제에서 묻는 조건을 정확히 해석합니다.
- 특히, 최소 기준인지("…이상"), 최대 한도인지("…초과할 수 없다"), 예외 조항인지(특정 전공/휴학 등)를 명확히 정리합니다.
- 일반 상식이나 다른 학교 사례를 사용하지 말고, **EWHA.pdf 문장만으로** 논리를 전개합니다.

단계 3: [선택지 비교]
- 각 선택지 {answer_options}가 EWHA.pdf 조문과 어떻게 다른지/같은지 하나씩 비교합니다.
- 조문과 **가장 일치하는 선택지** 또는
  부정형 문제의 경우 **조문에 규정되지 않은/예외에 해당하는 선택지**를 찾습니다.
- 시험 문제답게, 여러 해석이 가능해 보이면
  ① 조문과 정반대인 선택지,
  ② 조문에 전혀 언급되지 않은 내용을 추가로 가정하는 선택지부터 오답으로 처리합니다.

최종 답변: [ANSWER]: (X)  (X는 {answer_options} 중 하나)""")
```

- **Chain of Thought(CoT) Output**
  - 3-stage output structure
  - Isolation of final answer
  - Standardized format:
    [ANSWER]: (A/B/C/D)
    → 👍**post-processing**

Problem-Solving Prompt Templates: English (1)


```
    else:
        few_shot_text = get_few_shot_text(primary_cat, lang="en")

        return PromptTemplate.from_template(f"""You are an expert at answering
difficult multiple-choice questions (MMLU-Pro style), specializing in
**{primary_cat.upper()}**.

Your job:
- Use the **academic context** below as your PRIMARY source.
- BUT you may also use your own background knowledge **to fill in gaps or break ties**.
- If the context gives a **clear definition, rule, or condition** related to
**{primary_cat.upper()}**,
  you MUST follow that strictly, even if it conflicts with your memory.

Question: {{question}}

Academic Context: {{context}}

{few_shot_text}
```

- **Role**
  Confined to {Primary Category} Specialist
- **Single Domain Enforcement**
  Exclusive recognition of {Primary Category}, ignoring secondary inputs/categories

Problem-Solving Prompt Templates: English (2)


```
Guidelines:
1. First, extract the key definitions, rules, or facts from the context that
specifically relate to **{primary_cat.upper()}**.
2. Then check how each option {answer_options} matches or conflicts with those facts.
3. If the context is vague or incomplete, rely ONLY on standard **textbook knowledge of
{primary_cat.upper()}** to choose. Do not mix concepts from other disciplines.
4. Prefer the option that:
   - Best matches the core **{primary_cat.upper()}** principle in the context, AND
   - Sounds like a typical textbook or exam answer.
5. Avoid speculative or overly creative interpretations.

Format your response exactly as follows:

Step 1: [Key concepts and relevant information]
Step 2: [Understanding the question's requirements]
Step 3: [Option analysis]
Final Answer: [ANSWER]: (X) where X is one of {answer_options}""")
```

- **Hierarchical RAG Strategy**
  - **2-Tier knowledge hierarchy**
    - Priority Rule: Absolute precedence of Explicit Context
    - Fallback Mechanism: Restricted to standard {Primary Category} textbook knowledge
  - **Option Selection Criteria**
    : Alignment with typical textbook answers
- **Chain of Thought(CoT) Output**
  - 3-stage output structure
  - Isolation of final answer
  - Standardized Format: **[ANSWER]: (A~J)**
    → 👍**post-processing**

Answer Extraction Logic


```python
def extract_answer(response_text: str):
    if re.search(r'[가-힣]', str(response_text)):
        valid_chars = "ABCD"
    else:
        valid_chars = "ABCDEFGHIJ"

    patterns = [
        r"최종 답변:[\s\*]*\[ANSWER\]:[\s\*]*[\(\[\{]?([A-J])",
        r"\*\*ANSWER\*\*:[\s\*]*[\(\[\{]?([A-J])",
        r"\[ANSWER\]:[\s\*]*[\(\[\{]?([A-J])",
        r"Final Answer:[\s\*]*[\(\[\{]?([A-J])",
        r"Answer:[\s\*]*[\(\[\{]?([A-J])",
        r"ANSWER:[\s\*]*[\(\[\{]?([A-J])",
        r"정답:[\s\*]*[\(\[\{]?([A-J])",

        # LaTeX 포맷
        r"\\boxed\s*\{?\s*\(?([A-J])",

        # 문장 맨 끝에 답이 나오는 경우
        r"[\s\*]*[\(\[\{]([A-J])[\)\]\}][\s\*]*$"
    ]

    candidates = []

    for pattern in patterns:
        for match in re.finditer(pattern, response_text, re.IGNORECASE):
            ans = match.group(1).upper()
            if ans in valid_chars:
                # (등장 위치 index, 정답 문자) 저장
                candidates.append((match.start(), ans))

    if candidates:
        candidates.sort(key=lambda x: x[0]) # 위치순 정렬
        return candidates[-1][1] # 가장 뒤에 있는 정답 채택

    return None
```

Maximized coverage of diverse cases
via multiple regular expressions

Stage 1: Classification


```
classifier_chain = get_category_classifier_chain_multilabel()
pred_primary = []
pred_secondary = []
detected_languages = []

for question in tqdm(prompts, desc="Classifying"):
    if re.search(r'[가-힣]', str(question)):
        lang = "ko"
        p_cat, s_cat = "korean_regulation", None
    else:
        lang = "en"
        try:
            raw_out = classifier_chain.invoke({"question": question}) # LLM 호출
            p_cat, s_cat = parse_multilabel_output(raw_out) # 파싱
        except Exception as e:
            print(f"⚠️ Classification Error: {e}")
            p_cat, s_cat = "general", None

    detected_languages.append(lang)
    pred_primary.append(p_cat)
    pred_secondary.append(s_cat)


df['pred_primary'] = pred_primary
df['pred_secondary'] = pred_secondary
df['language'] = detected_languages
```

If the question is **in Korean**,
the category is **directly set to korean_regulation**
without LLM intervention.

If the question is **not in Korean**,
the **LLM is used for classification**,
and the output is parsed and saved.

Stage 2: Problem Solving (1) Retrieval


```python
results = []

for idx, row in tqdm(df.iterrows(), total=len(df), desc="Processing"):
    question = row['prompts']
    p_cat = row['pred_primary']
    s_cat = row['pred_secondary']
    language = row['language']
    # Retriever 설정
    if language == "ko":
        retriever = ewha_hybrid_retriever
        num_choices = 4
        search_query = create_enhanced_query(question)
    else:
        retriever = academic_hybrid_retriever
        num_choices = 10
        search_query = question

    # Context 검색
    try:
        retrieved_docs = retriever.invoke(search_query)
        context_text = "\n\n".join([doc.page_content for doc in retrieved_docs])
    except Exception as e:
        print(f"Retrieval Error at idx {idx}: {e}")
        context_text = ""
```

For **Korean** Questions,
utilize the **ewha_hybrid_retriever**

For **English** Questions,
utilize the **academic_hybrid_retriever**

Document Processing

- Receive the query and **retrieve the most relevant documents**
- Concatenate the content of the extracted documents into a single string, stored as **context_text**

Stage 2: Problem Solving (2) Generation


```python
# Mixture Prompt 생성
prompt_template = get_focused_rag_prompt(p_cat, s_cat, num_choices)
rag_chain = prompt_template | llm_reasoning | StrOutputParser()

# 답변 생성
try:
    response_text = rag_chain.invoke({
        "question": question,
        "context": context_text
    })
except Exception as e:
    print(f"Error in answer-generation: {e}")
    response_text = "Error"

# 정답 추출
generated_answer = extract_answer(response_text)

# 결과 수집
results.append({
    "question": str(question),
    "your_answer": generated_answer
})
```

- **Prompt completion** utilizing predicted categories
- **RAG Chain Setup**

RAG Chain Execution

Answer extraction based on LLM output

Accuracy: MMLU-pro Sampled Dataset


```
================================================================================
                          FINAL ANALYSIS REPORT
================================================================================
1. Total Questions              : 100
2. Answer Accuracy (RAG)        : 63.00% (63/100)
3. Classification Accuracy (LLM): 91.00% (91/100)
--------------------------------------------------------------------------------
[Category Classification Detail]
pred_primary   business  general  history  law  philosophy  psychology
true_category
business             20        0        0    0           0           0
history               0        1       16    2           1           0
law                   0        0        0   19           1           0
philosophy            0        0        2    1          17           0
psychology            1        0        0    0           0          19
================================================================================
```

```
================================================================================
📊 [오답 통계] True Category vs Pred Category (어디서 많이 틀렸나?)
================================================================================
pred_primary   business  history  law  philosophy  psychology
true_category
business              7        0    0           0           0
history               0        4    1           0           0
law                   0        0   11           0           0
philosophy            0        0    0           6           0
psychology            0        0    0           0           8
--------------------------------------------------------------------------------
 * 행(Row): 실제 정답 카테고리
 * 열(Col): AI가 잘못 예측한 카테고리 (또는 분류는 맞았는데 답이 틀린 경우)
================================================================================
```

```
================================================================================
📊 [오답 통계] true_category vs pred_secondary (주 카테고리 분류도 틀린 오답에 대해서)
================================================================================
pred_secondary  history  philosophy  psychology
true_category
history               1           1           1
philosophy            0           3           0
--------------------------------------------------------------------------------
 * 행(Row): 실제 정답 카테고리
 * 열(Col): AI가 보조 카테고리로 예측한 카테고리
================================================================================
```

HuggingFace 🤗 **TIGER-Lab/MMLU-Pro**

Business, History, Law, Philosophy, and Psychology

Random extraction of **20 questions per category → shuffle**

Classification Performance Analysis (True vs. Primary Category)

classification or parsing **failure → general**

Error Analysis (Primary Category)

- Minimal off-diagonal errors
- Absence of ‘general’ category in incorrect answers

→ **Negligible impact** of category misclassification on overall error rate

Error Analysis (Secondary Category)

Low count of secondary predictions relative to total errors

= Indication of **high confidence** in Primary Category predictions

Accuracy: testset.csv


```
================================================================
                    최종 정확도 분석
================================================================
전체 정확도:                88.00% (44/50)

한국어 (EWHA, 4지선다):    100.00% (25/25)
영어 (MMLU-Pro, 10지선다): 76.00% (19/25)

【 MMLU-Pro 카테고리별 성능 】
LAW          :  66.67% (4/6)
PSYCHOLOGY   :  85.71% (6/7)
BUSINESS     :  50.00% (1/2)
PHILOSOPHY   :  75.00% (3/4)
HISTORY      :  83.33% (5/6)
================================================================
```

testset.csv

Enhancement of ewha.pdf parsing logic
→ 100% Accuracy ★

Category-based assessment
via Stage 1 predictions

[Ground Truth: Distribution of Category]
LAW: 6
PSYCHOLOGY: 6
BUSINESS: 2
PHILOSOPHY: 4
HISTORY: 7

Contributions Across Project Stages

Eunna	• Data Embedding: Law & History Domains • Prompt Engineering: Instruction design • Model Optimization: Hyperparameter Grid-Search • Documentation: Presentation Structure
Jungyeon	• Data Embedding: Wikipedia & Philosophy Domains • Data Parsing: University Regulations (ewha.pdf) Processing • Prompt Engineering: One-shot Example Construction • Documentation: Presentation Structure
Seonyeong	• Data Embedding: Business & Psychology Domains • Model Implementation: Category Classifier LLM • Code Integration • Documentation: Presentation Structure

Thank you for listening